Secure Authentication System and Research AI Workspace

Objective 1 and Objective 2 Project Report

Submitted By

Mayank

Roll Number: CD1915

Department of Information Technology

Rungta College of Engineering and Technology

Under the Guidance of

Prof. D.K. Sir

NIT Jamshedpur

Academic Year: 2026

Certificate

This is to certify that Mayank, Roll Number CD1915, from the Department of Information Technology, Rungta College of Engineering and Technology, has successfully completed the project titled **Secure Authentication System and Research AI Workspace**. The project was carried out under the guidance of Prof. D.K. Sir, NIT Jamshedpur.

Signature of Guide: ________________________

Signature of Student: ________________________

Acknowledgement

I would like to express my sincere gratitude to Prof. D.K. Sir from NIT Jamshedpur for his guidance and support throughout this project. I am also thankful to Rungta College of Engineering and Technology for providing me the opportunity to develop this project.

This project helped me gain practical knowledge of frontend development, backend API development, secure authentication, JWT implementation, MongoDB Atlas database connectivity, PDF upload handling, AI chatbot integration, GitHub version control and Render cloud deployment.

Abstract

This project presents the design and development of a secure authentication system integrated with an AI powered research workspace. The project is divided into two major objectives. Objective 1 focuses on the implementation of secure user authentication using registration, login, password protection, JWT based authorization and MongoDB database connectivity. Objective 2 extends the system by introducing a research dashboard where authenticated users can upload PDF files, view uploaded documents, delete documents and interact with an AI chatbot.

The system follows a full-stack architecture consisting of HTML, CSS and JavaScript on the frontend, Node.js and Express.js on the backend, MongoDB Atlas as the cloud database, and Render as the deployment platform. The project demonstrates practical usage of REST APIs, database models, middleware, authentication tokens, file upload systems and cloud hosting.

Contents

Chapter 1

Introduction

In modern web applications, user authentication and secure data access are essential components. Any application that manages users, documents or private information requires a reliable login and registration system. Without authentication, unauthorized users may access sensitive data or misuse system resources.

This project combines secure authentication with an AI enabled research workspace. The first part of the project builds a secure authentication system where users can register and login. The second part adds a dashboard where users can manage PDF files and interact with an AI chatbot.

The system is designed as a practical full-stack application. It uses frontend technologies for user interface design, backend technologies for API logic, MongoDB Atlas for database storage and Render for cloud deployment.

1.1 Need of the Project

The need for this project arises from the increasing demand for secure web platforms that can also provide intelligent assistance. Students, researchers and developers often need systems where they can store documents, manage files and ask questions from uploaded content. Combining authentication with AI makes the platform more useful and secure.

1.2 Main Objectives

- To develop a secure user registration system.
- To implement login authentication using JWT.
- To connect the backend with MongoDB Atlas.
- To create a protected dashboard for authenticated users.

- To upload and manage PDF documents.
- To add AI chatbot functionality.
- To deploy the application on Render.
- To maintain project source code using GitHub.

Chapter 2

Objective 1: Secure Authentication System

2.1 Overview

Objective 1 focuses on building a secure authentication system. The authentication system allows new users to create accounts and existing users to login securely. After successful login, a JWT token is generated by the backend and stored in the browser local storage. This token is used to verify whether the user is authorized to access the dashboard.

2.2 Problem Statement

Many simple web applications do not provide strong authentication. Storing passwords without encryption, allowing direct dashboard access and not validating users properly can create serious security risks. The problem is to develop a system where only authenticated users can access protected resources.

2.3 Features

- User registration with name, email and password.
- Login using email and password.
- Password hashing using backend security methods.
- JWT token generation after login.
- Protected dashboard access.

- MongoDB Atlas database integration.
- REST API based frontend-backend communication.

2.4 Authentication Workflow

The authentication process works in the following steps:

1. User opens the frontend homepage.
2. User clicks the register button.
3. User enters name, email and password.
4. Frontend sends registration data to backend API.
5. Backend validates the input.
6. User information is stored in MongoDB.
7. User opens login modal.
8. User enters email and password.
9. Backend verifies credentials.
10. JWT token is created.
11. Frontend stores the token in local storage.
12. User is redirected to the protected dashboard.

2.5 Advantages of JWT

JWT stands for JSON Web Token. It is used to securely transfer user identity information between client and server.

- It is lightweight.
- It supports stateless authentication.
- It can store user identity securely.
- It is suitable for REST APIs.
- It helps protect dashboard routes.

Chapter 3

Objective 2: Research AI Workspace

3.1 Overview

Objective 2 extends the authentication system by adding an AI powered dashboard. After login, users can access a research workspace where they can upload PDF files, view documents, delete documents and communicate with an AI chatbot.

3.2 Problem Statement

Research documents are often long and difficult to analyze manually. Users may need quick answers from uploaded documents or general AI assistance. The objective is to provide a simple dashboard that supports PDF management and AI based interaction.

3.3 Features

- Protected research dashboard.
- PDF upload option.
- Uploaded PDF list.
- Open PDF button.
- Delete PDF button.
- AI chatbox.
- Document based question answering support.
- Backend file processing.

3.4 PDF Upload Workflow

1. User logs into the system.
2. User opens the dashboard.
3. User clicks Upload PDF.
4. User selects a PDF file.
5. Frontend sends the PDF using FormData.
6. Backend receives the file using Multer.
7. File is stored inside uploads folder.
8. Document metadata is saved in MongoDB.
9. Uploaded PDF appears in the View PDFs section.

3.5 AI Chat Workflow

1. User opens AI Chat section.
2. User enters a question.
3. Frontend sends the question to backend API.
4. Backend processes the question.
5. AI service generates response.
6. Response is displayed in the chatbox.

Chapter 4

Technology Stack

4.1 Frontend

The frontend is developed using HTML, CSS and JavaScript.

Technology	Usage
HTML	Used to create the structure of homepage, login form, registration form and dashboard.
CSS	Used for styling, animations, layout, buttons, cards and responsive design.
JavaScript	Used for form handling, API calls, JWT storage, dashboard navigation and chat interaction.

4.2 Backend

The backend is developed using Node.js and Express.js.

Technology	Usage
Node.js	Runtime environment for backend JavaScript.
Express.js	Used to create REST API routes.
Mongoose	Used to connect backend with MongoDB Atlas.
JWT	Used for secure token based authentication.
Multer	Used for PDF file upload handling.

4.3 Database

MongoDB Atlas is used as the cloud database. It stores user details, uploaded document metadata and application records.

4.4 Deployment

Render is used to deploy the backend service. GitHub is used for version control and automatic deployment workflow.

Chapter 5

System Architecture

5.1 Architecture Flow

User Interface

$\downarrow$

HTML CSS JavaScript Frontend

$\downarrow$

Fetch API Requests

$\downarrow$

Node.js Express Backend

$\downarrow$

MongoDB Atlas Database

$\downarrow$

AI Chat Service

5.2 Explanation

The user interacts with the frontend. The frontend sends HTTP requests to the backend. The backend handles authentication, file upload and chat logic. MongoDB stores user and document data. Render hosts the backend online so that the frontend can communicate with it through a public API URL.

Chapter 6

Implementation Details

6.1 User Registration

The registration feature collects name, email and password from the user. The frontend sends the data to the backend register endpoint. The backend checks whether the user already exists. If the user does not exist, a new user is created in MongoDB.

6.2 User Login

The login system verifies the user email and password. If credentials are valid, the backend creates a JWT token and sends it to the frontend. The token is stored in local storage.

6.3 Protected Dashboard

The dashboard checks whether a JWT token exists in local storage. If no token is found, the user is redirected to the login page. This prevents unauthorized dashboard access.

6.4 PDF Upload

PDF upload is handled using Multer middleware. The file is stored in the uploads folder and document details are saved in MongoDB. The dashboard fetches and displays uploaded files.

6.5 PDF Delete

The delete feature allows users to remove uploaded PDFs. The frontend sends a DELETE request to the backend. The backend removes the document record from MongoDB and deletes the file if available.

6.6 AI Chatbox

The AI chatbox allows users to ask questions. JavaScript captures the message and sends it to the backend. The backend returns an AI generated response, which is displayed in the chat interface.

Chapter 7

Project Implementation Screenshots

7.1 Home Page Interface

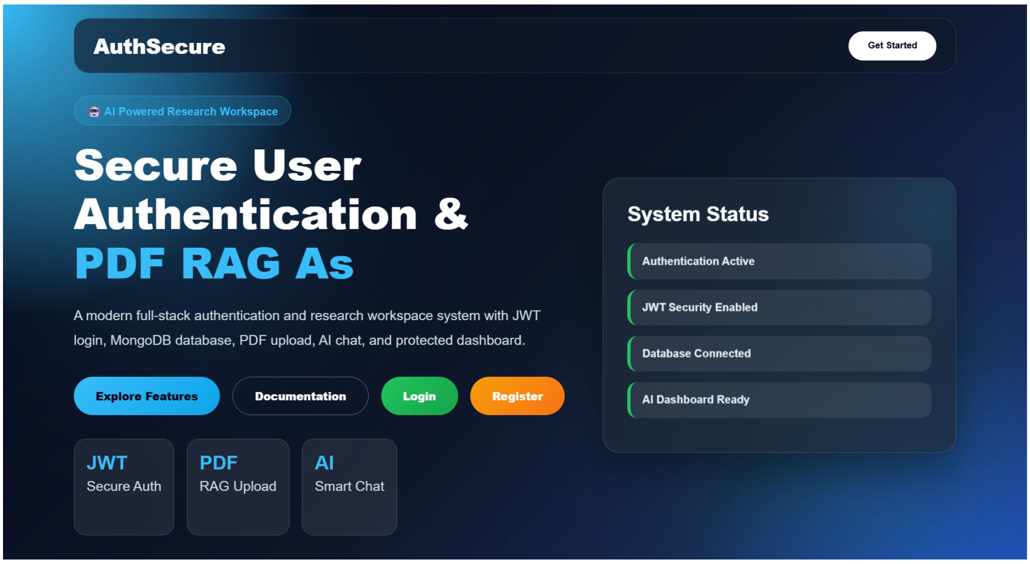

Figure 7.1: Secure Authentication Homepage Interface

7.2 User Registration Page

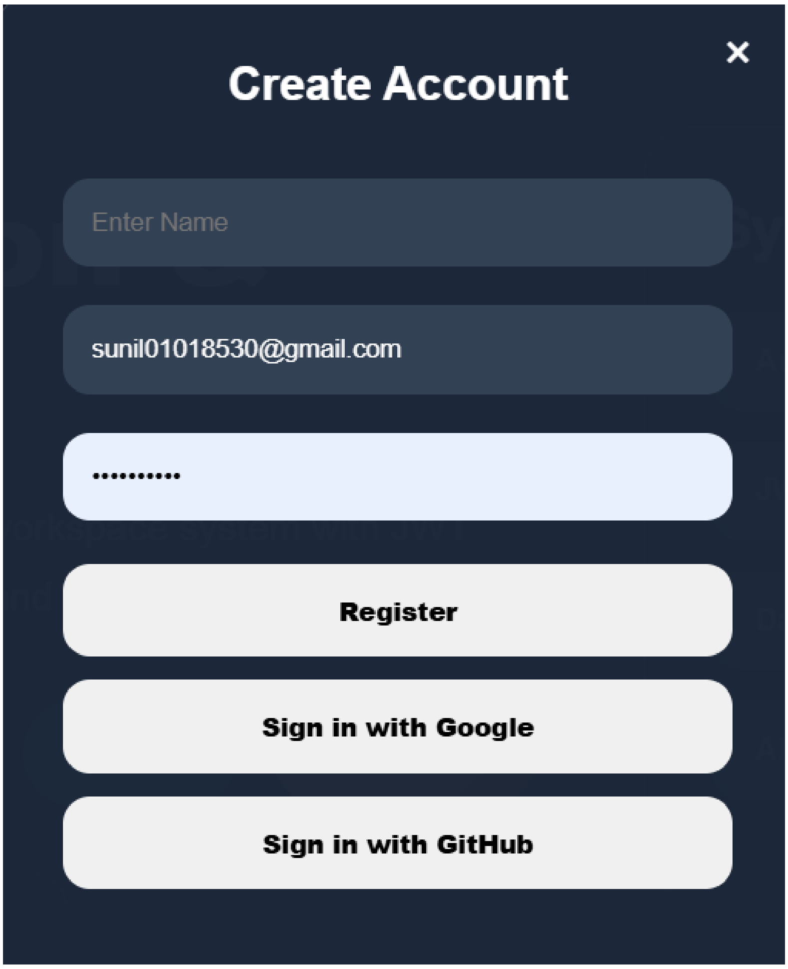

Figure 7.2: User Registration Interface

7.3 Login Page

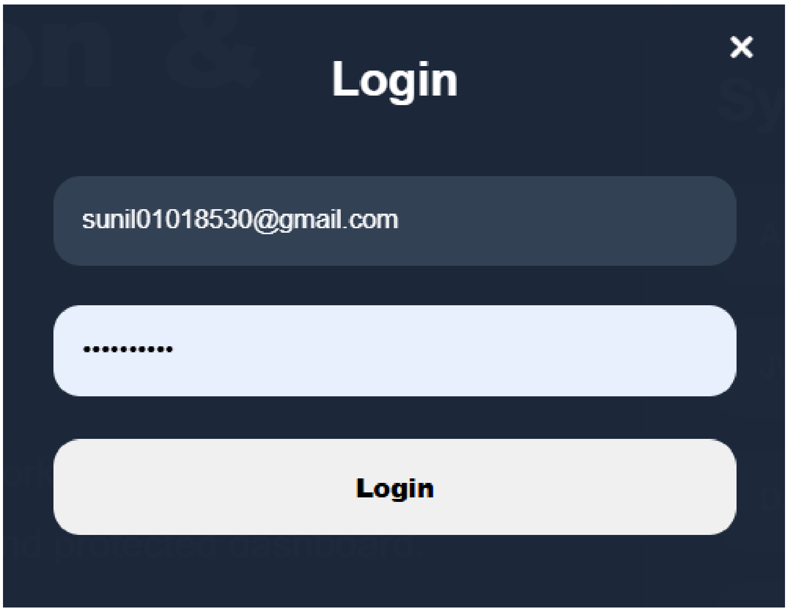

Figure 7.3: User Login Interface

7.4 Research Workspace Dashboard

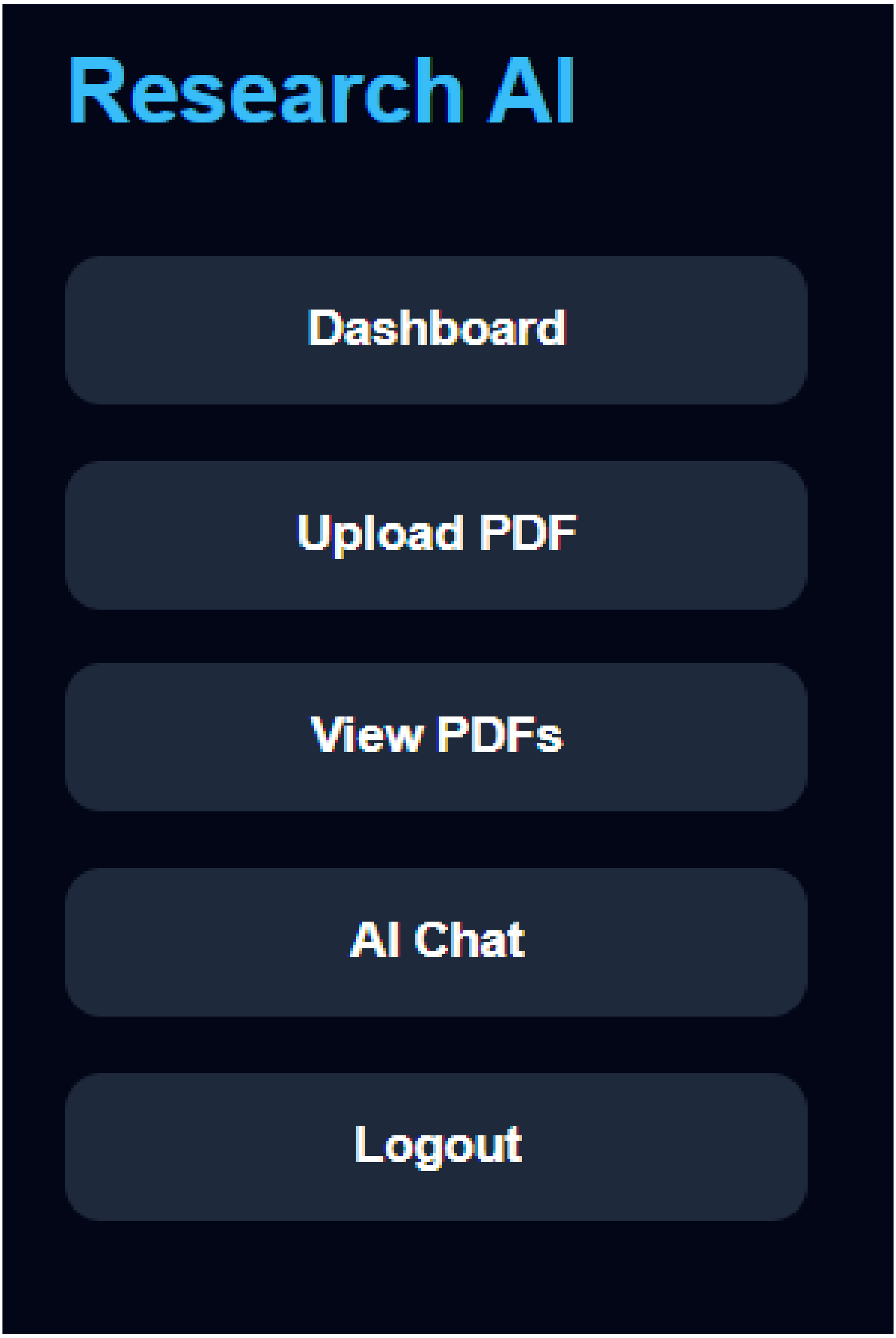

Figure 7.4: Research Workspace Dashboard

7.5 PDF Upload Implementation

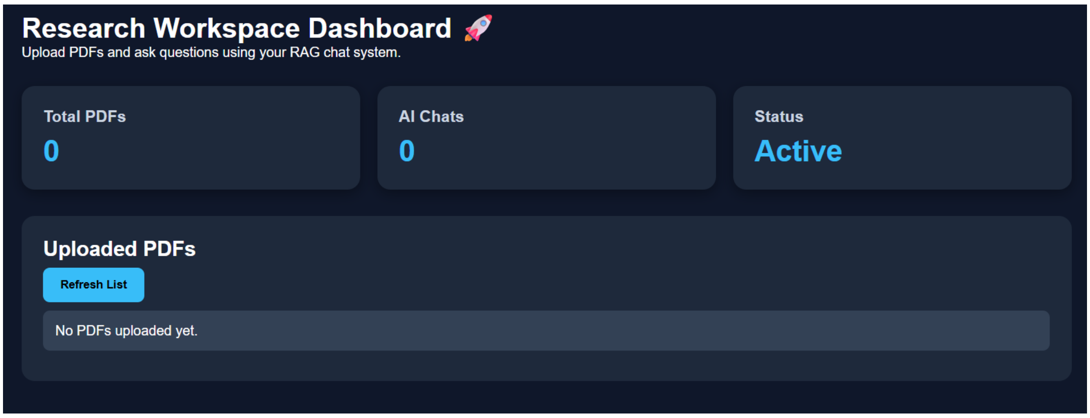

Figure 7.5: PDF Upload Interface

7.6 AI Chat Implementation

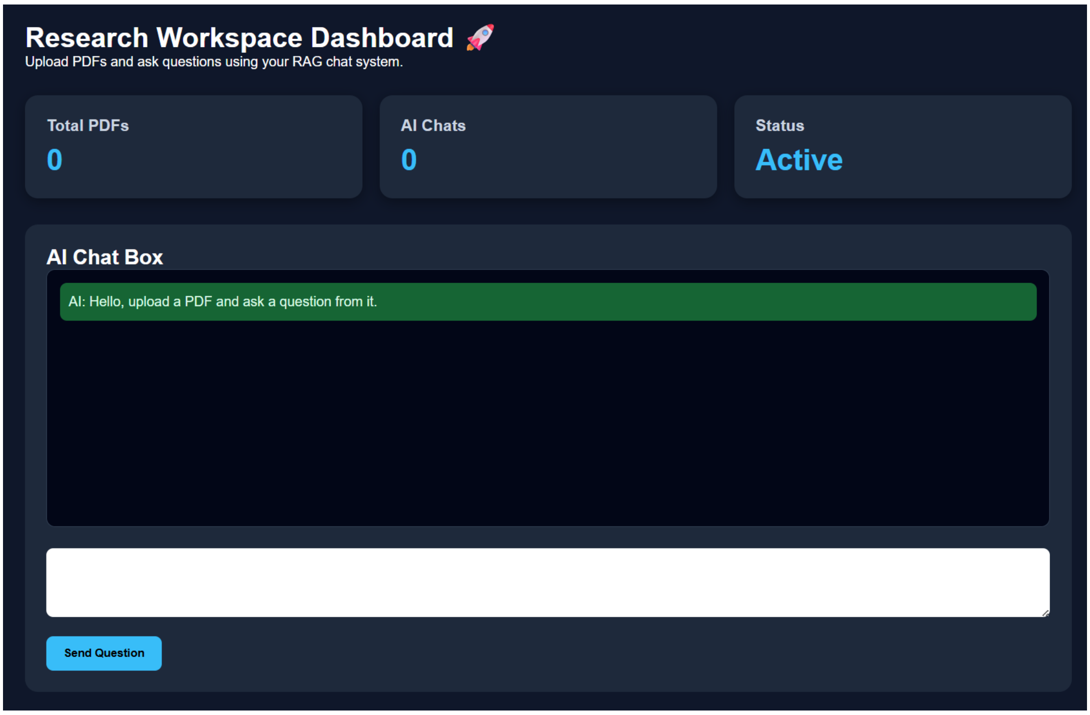

Figure 7.6: AI Chatbot Interface

7.7 MongoDB Atlas Database

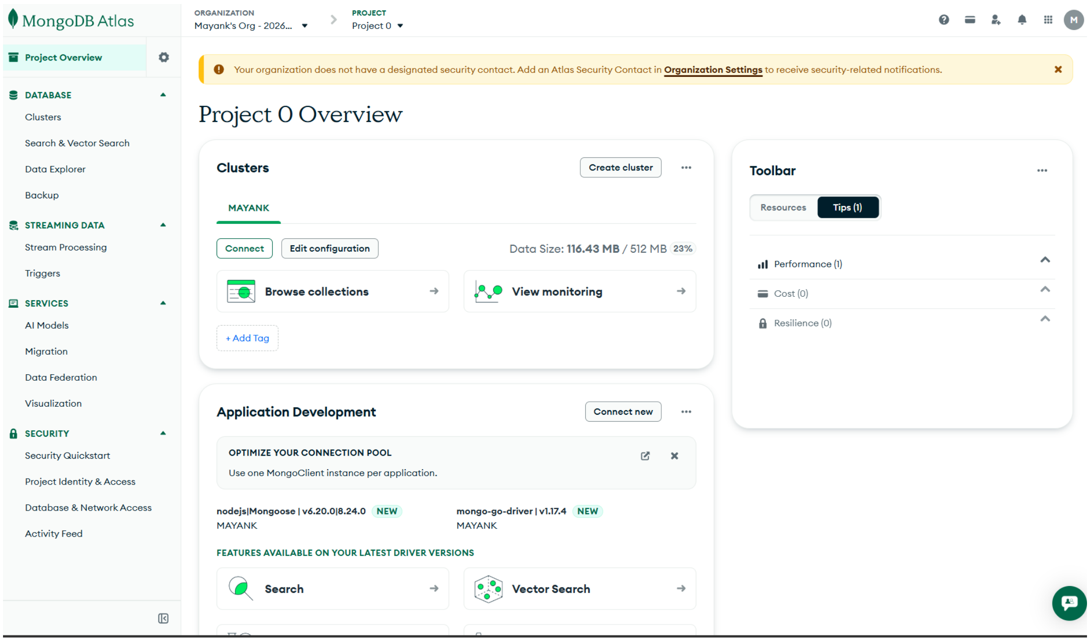

Figure 7.7: MongoDB Atlas Database

7.8 GitHub Version Control

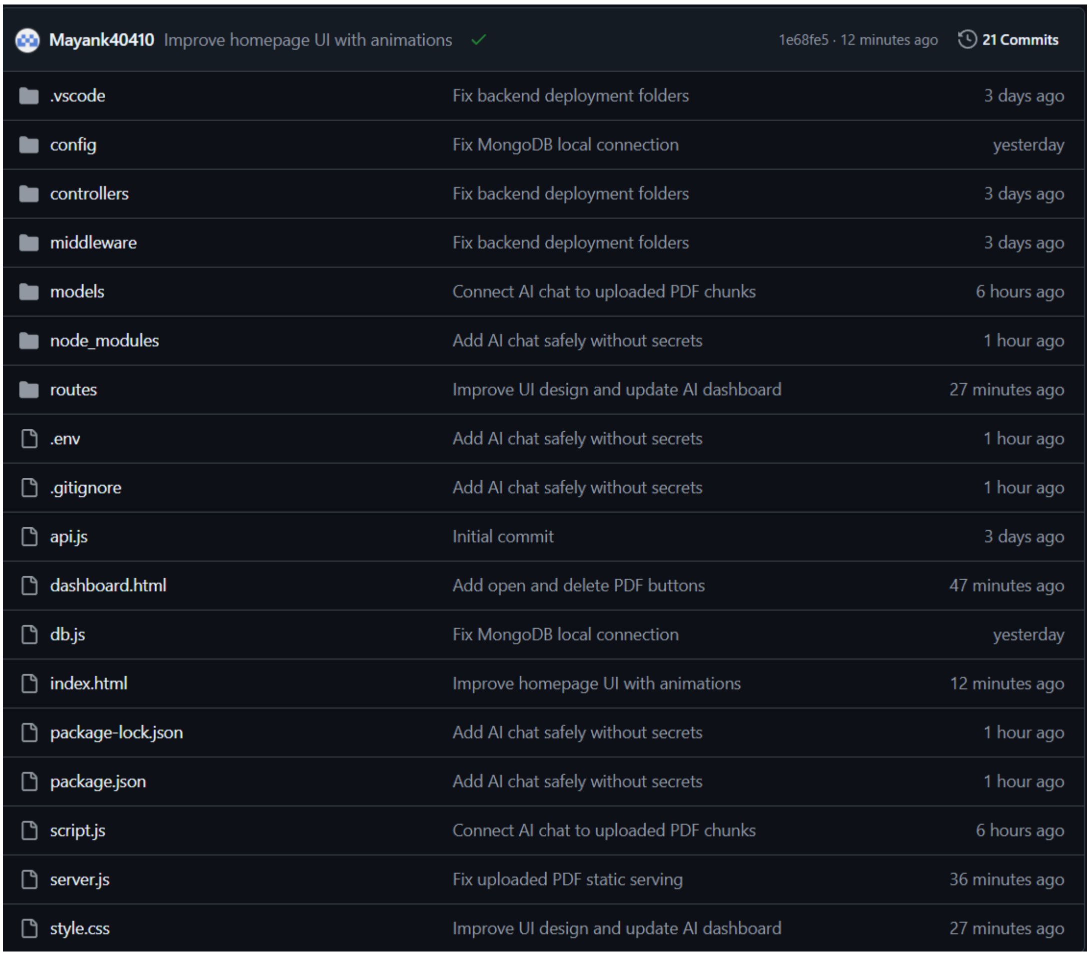

Figure 7.8: GitHub Repository

7.9 Render Deployment

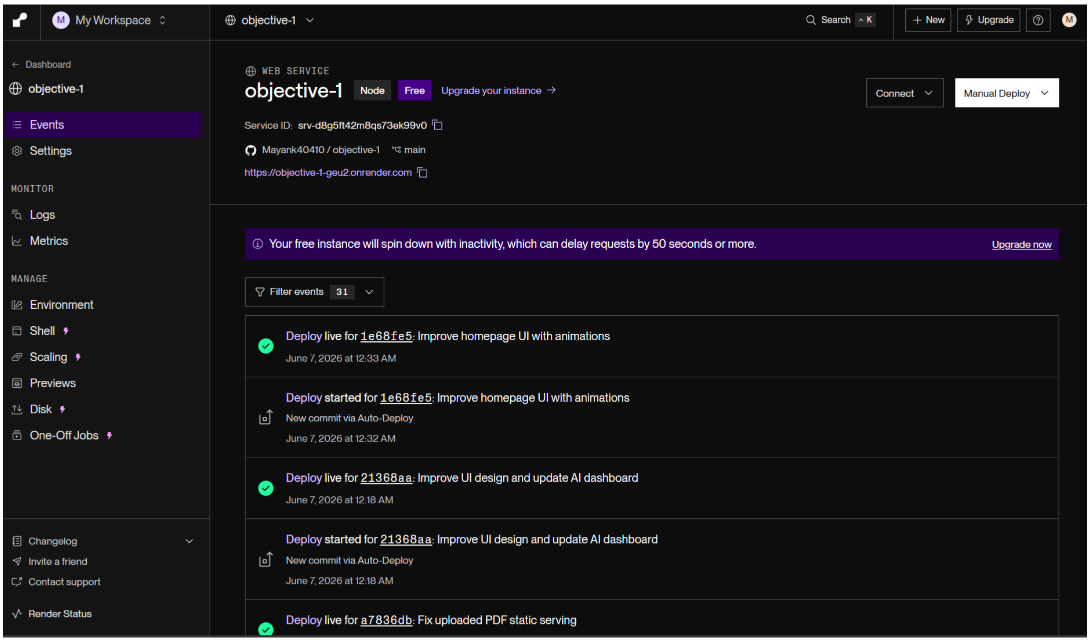

Figure 7.9: Render Deployment Dashboard

Chapter 8

Testing

8.1 Registration Testing

The registration form was tested by entering valid name, email and password. The backend successfully stored the user in MongoDB.

8.2 Login Testing

The login feature was tested using registered email and password. The system generated a JWT token and redirected the user to dashboard.

8.3 PDF Upload Testing

PDF upload was tested by uploading a sample document. The document appeared in the uploaded PDF list.

8.4 AI Chat Testing

The AI chatbox was tested by asking sample questions. The chatbot displayed responses inside the dashboard.

Chapter 9

Result

The project successfully achieved the planned objectives.

- Secure authentication system completed.
- JWT based session handling implemented.
- MongoDB Atlas connected successfully.
- Protected dashboard created.
- PDF upload and management implemented.
- AI chatbot interface created.
- GitHub repository maintained.
- Backend deployed on Render.

Chapter 10

Conclusion

This project successfully demonstrates a secure authentication system and AI powered research workspace. Objective 1 helped in understanding authentication, backend API development, JWT security and database connectivity. Objective 2 helped in understanding PDF upload, dashboard design, AI chat integration and cloud deployment.

The system provides a practical example of modern full-stack development and can be extended further for real-world usage.

Chapter 11

Future Scope

Future improvements include:

- Role based access control.
- Email verification.
- Password reset system.
- Advanced RAG using vector database.
- User specific document storage.
- Admin dashboard.
- Better AI response accuracy.
- Improved UI animations.
- Document summary generation.

Chapter 12

References

- Node.js Documentation
- Express.js Documentation
- MongoDB Atlas Documentation
- JWT Documentation
- Render Deployment Documentation
- GitHub Documentation
- OpenAI API Documentation